

KOZALAK

Holografik Bellek Kasası & Güvenli Dağıtım Motoru

Mimari & Geliştirme: codebygunes

Sürüm 1.0.0 | Kapsamlı Kurumsal Dokümantasyon
(V2)

1. Yönetici Özeti ve Mimari Vizyon

KOZALAK (Thread-Split Enclave), modern siber güvenlik tehditlerine karşı "Fail-Secure" (Güvenli Çöküş) prensibiyle sıfırdan inşa edilmiş, düşük seviyeli (low-level) bir bellek yönetimi ve kriptografik veri dağıtım kütüphanesidir.

Monolitik bellek bloklarının aksine KOZALAK, veriyi **Shamir's Secret Sharing (GF257)** algoritması ile matematiksel olarak parçalar ve farklı işçi (worker) thread'lere böler. Sistem aktif olduğu sürece, şifrenin tamamı hiçbir thread'in belleğinde tek parça halinde bulunmaz.

2. Kapsamlı Güvenlik Katmanları ve Yaşam Döngüsü

2.1. Donanımsal Mühürleme (Hardware Sealing)

Sisteme giren veri, sunucunun donanımsal kimliği (Linux `/etc/machine-id`, Windows `uuid`) ile mühürlenir. RAM dökümü çalınsa dahi veri yalnızca üretildiği fiziksel CPU üzerinde çözülebilir.

2.2. Daemon Swap & Hareketli Hedef Mimarisi

İşçi thread'lerin yanında, içi sahte verilerle dolu Bal Küpü (Honeypot) thread'leri çalışır. Arka plandaki Daemon Thread, her 50 milisaniyede bir parçaları takas eder (Swap). Bu sayede statik bellek analizi ve adres kancalama (Hooking) imkansızlaşır.

2.3. Zehirli Hap (Poison Pill) & Call-Stack Analizi

Çalışma zamanında çağrı yığını (Call-Stack) denetlenir. `ptrace`, `frida`, `gdb` veya Python'dan sahte `eval/exec` çağrıları yakalandığında "Zehirli Hap" tetiklenerek anında sistem kilitlenir.

2.4. Yaşam Döngüsü ve Otomatik Asitle Yıkama (Zero-Out)

Sistemin güvenliği sadece dış saldırılara karşı değil, olağan yaşam döngüsünün (Lifecycle) sonuna da entegredir. İşlem tamamlanıp Python Garbage Collector nesneyi yok ettiğinde veya Rust tarafında `Drop` metodu çalıştığında, RAM'deki tüm parçalar derleyici optimizasyonlarını aşacak şekilde `ptr::write_volatile` ile sıfırlanır (Zero-Out). Sistem geride hiçbir veri kalıntısı (Data Remanence) bırakmaz.

3. Çekirdek Bileşenler ve Modül Yapısı

Bileşen / Modül	Teknik Sorumluluk
<code>shamir_share.rs</code>	Sırrın Galois Field (GF257) üzerinde matematiksel olarak parçalanması ve birleştirilmesi.
<code>memory_manager.rs</code>	İşletim sistemine RAM'in diske yazılmamasını emreden <code>mlock</code> / <code>VirtualLock</code> yönetimi.
<code>hardware_sealer.rs</code>	Platform bağımlı donanım kimliği üretimi ve XOR şifreleme/çözme.
<code>call_stack_inspector.rs</code>	Çalışma anında yetkisiz kanca girişimlerini tespit edip Zehirli Hap döngüsünü tetikleyen modül.
<code>thread_pool.rs</code>	mpsc kanalları üzerinden komut yürüten thread ve daemon mimarisinin orkestrasyonu.
<code>wasm_edge.rs</code>	Tarayıcı ortamı (Web Worker) için güvenli işlem imkanı sunan FFI köprüsü.

4. API Entegrasyonu ve Kullanım Akışı

KOZALAK, C-FFI (Foreign Function Interface) katmanı sayesinde farklı dillerle (Python, C++) sorunsuz çalışır. `enclave_api.py` modülü üzerinden tipik bir akış şu şekildedir:

- **Başlatma:** `vault = HolographicVault(secret=db_secret, num_threads=5, threshold=3)` çağrısıyla veri C kütüphanesine iletilir ve bellekte anında parçalanır. Orijinal değişken (string) ortadan kalkar.
- **Güvenli Yürütme:** `vault.execute_with(connect_to_yanki_db)` metodu ile kasa mikrosaniyeler içinde kilidini açar, parçaları birleştirir ve referans pointer olarak Python fonksiyonuna iletir.
- **Kapanış:** Callback (fonksiyon) çalışmasını bitirdiği an süreç sonlanır ve RAM Zero-Out protokolüyle ezilir.

5. Red Team Simölasyonları ve Test Raporları

Sistem, en zorlu siber güvenlik simölasyonlarından (Red Team) başarıyla geçmiştir:

Test 1: Polimorfik Kaydırma (Hareketli Hedef)

Daemon Thread 300ms boyunca parçaları saniyede 20 kez havada takas etmesine rağmen veri sıfır veri kaybı ile güvenle birleştirilmiştir.

Test 2: Donanım Mührü ve RAM Dökümü Hırsızlığı

Saldırgan RAM dökümünü çalıp Shamir parçalarını kendi bilgisayarında birleştirmeye çalışmıştır. Orijinal sunucunun (lokal CPU) donanım imzası eşleşmediği için elde edilen veri tamamen anlamsız gürültüden ibaret kalmış ve zafiyet engellenmiştir.

Test 3: Eşzamanlılık (Concurrency) ve Stres Testi

10 farklı işlemin aynı mikrosaniyede Mutex kilidine saldırdığı senaryoda çakışma (Race Condition) yaşanmamış, mimari kusursuz çalışmıştır.

Test 4: Çağrı Yığını (Call-Stack) RCE / Eval Denemesi

Sisteme dışarıdan sahte bir Python `eval` sızma denemesi yapıldığında, Call-Stack Inspector bu şüpheli çağrıyı anında fark etmiş ve sırları teslim etmemek için kütüphanenin kendi sürecini imha etmesini sağlamıştır.

Test 5: DMA Donanım Kancası ve Honeypot İhlali

İşçi thread'lerden birine dışarıdan sahte bir parça (Honeypot/Tuzak) enjekte edilmiştir. Sistem "MAGIC_MARKER" imzasının bozulduğunu fark ettiği milisaniyede Zehirli Hap'ı tetiklemiş ve süreci çökertmiştir.

Edge Desteği (WASM)

Derlenen `thread_split_enclave.wasm` modülü sadece **560.49 KB** boyutundadır ve tarayıcı ortamını XSS saldırılarına karşı korumak üzere yüksek optimizasyonla çalışmaktadır.